MÌ AI · HỌC AI THEO CÁCH MÌ ĂN LIỀN

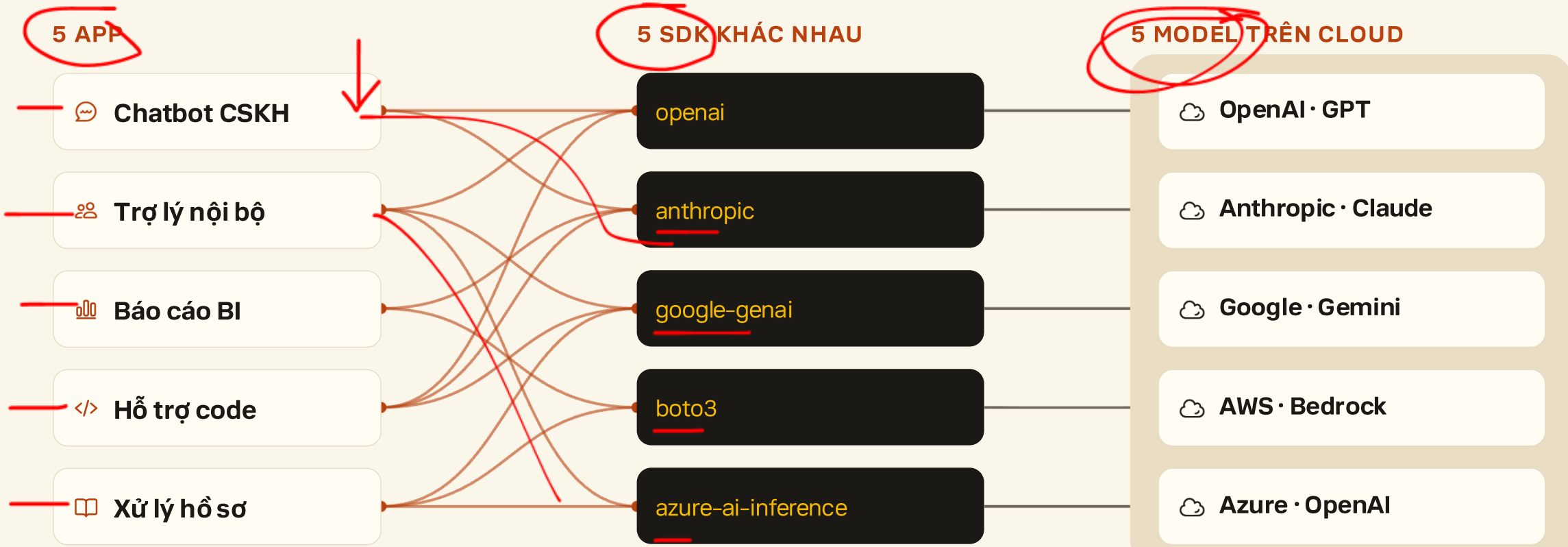
LiteLLM: từ cá nhân, đến trong doanh nghiệp

SDK + AI Gateway mã nguồn mở – gọi mọi model theo chuẩn OpenAI

Nguyễn Chiến Thắng · Founder Mì AI

HIỆN TRẠNG

5 app, 5 SDK, 5 model: một mạng nhện



VẤN ĐỀ

Dùng 5 model = 5 cơn đau đầu

5 SDK, 5 kiểu code

OpenAI một kiểu, Anthropic một kiểu, Gemini lại một kiểu khác.

Đổi model là sửa code

Muốn thử model mới phải viết lại phần gọi API và phần đọc kết quả.

Không biết tiêu bao nhiêu

Sếp hỏi tháng này tốn bao nhiêu tiền token, không ai trả lời được.

Provider sập là app sập

Một hãng lỗi hoặc hết quota là cả ứng dụng đứng theo.

TRƯỚC KHI CÓ LITELLM

Mỗi hãng một kiểu code

```python
# openai_app.py
from openai import OpenAI

client = OpenAI()
r = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user",
               "content": "Xin chào!"}],
)
print(r.choices[0].message.content)
```

```python
# anthropic_app.py
import anthropic

client = anthropic.Anthropic()
r = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=1024,       # bắt buộc!
    messages=[{"role": "user",
               "content": "Xin chào!"}],
)
print(r.content[0].text)   # khác format!
```

Thêm Gemini, Bedrock, Ollama… mỗi hãng lại **một SDK, một format riêng**. Code càng ngày càng rối.

LITELLM LÀ GÌ?

Ổ cắm du lịch đa năng cho LLM



100+

nhà cung cấp LLM

61.3k

GitHub stars (11/2026)

Self-hosted

chạy trên hạ tầng của bạn

HAI CÁCH DÙNG

SDK cho dev, Gateway cho tổ chức

</>

Python SDK

Import thư viện, gọi **completion()** ngay trong code.

- Dev cá nhân, prototype, notebook
- Có sẵn retry, fallback, tính chi phí

```
pip install litellm
```

AI Gateway (Proxy)

Một dịch vụ trung tâm, mọi app trong công ty gọi qua đây.

- Virtual key, ngân sách theo team
- Dashboard, log, guardrails

```
pip install 'litellm[proxy]'
```

Demo 1

Python SDK

Cài đặt · đổi model · streaming · chi phí · fallback

DEMO 1 · CHUẨN BỊ

Tra tên hãng và model ở đâu?

1 Tên hãng (provider)

docs.litellm.ai/docs/providers

Mỗi hãng một trang: tiền tố, biến môi trường cho key, ví dụ code.

Lưu ý: Gemini qua AI Studio là **gemini/**, qua Google Cloud là **vertex_ai/**.

2 Tên model

Tài liệu API của chính hãng

Phần sau dấu / là model ID hãng công bố. LiteLLM chỉ chuyển tiếp nguyên tên.

Hãng ra model mới: điền tên mới là gọi được.

3 Tra trong code

Bảng model đi kèm thư viện, cũng là bảng giá cho completion_cost().

```
from litellm import models_by_provider
print(list(models_by_provider))        # danh sách hãng
print(models_by_provider["anthropic"])  # model của 1 hãng
```

DEMO 1 · BƯỚC 1

Gọi model đầu tiên

quickstart.py

1. Cài thư viện litellm

2. Đặt API key qua biến môi trường

3. Gọi completion(): kết quả trả về theo chuẩn OpenAI

Quy tắc đặt tên model

`hãng/tên_model`

```
# pip install litellm
import os
from litellm import completion

os.environ["OPENAI_API_KEY"] = "sk-..."

response = completion(
    model="openai/gpt-4o",
    messages=[{"role": "user",
               "content": "Giải thích LLM trong 1 câu"}],
)
print(response.choices[0].message.content)
```

DEMO 1 · BƯỚC 2

Đổi model = đổi 1 chuỗi

multi_models.py

4 model, 1 câu hỏi, 0 dòng phải sửa

Chỉ cần đặt key: OPENAI_API_KEY, GEMINI_API_KEY

Ollama chạy trên máy, không cần key

```python
from litellm import completion

models = [
    "openai/gpt-4o",
    "anthropic/claude-sonnet-5-5",
    "gemini/gemini-2.5-flash",
    "ollama/llama3.2",        # chạy local, miễn phí
]
question = [{"role": "user", "content": "Phở hay bún chả?"}]

for m in models:
    r = completion(model=m, messages=question)
    print(f"--- {m}\n{r.choices[0].message.content}")
```

DEMO 1 · BƯỚC 3

Streaming và tính tiền tự động

`stream_cost.py`

stream=True: chữ hiện ra dần, trải nghiệm như ChatGPT

completion_cost(): tính tiền theo USD dựa trên bảng giá model LiteLLM duy trì sẵn

```python
from litellm import completion, completion_cost

msgs = [{"role": "user", "content": "Kể chuyện cười IT"}]

# 1) Streaming: chữ hiện ra dần như ChatGPT
stream = completion(model="openai/gpt-4o", messages=msgs,
                    stream=True)
for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="")

# 2) Biết ngay mỗi lần gọi tốn bao nhiêu
r = completion(model="anthropic/claude-sonnet-5-5", messages=msgs)
cost = completion_cost(completion_response=r)
print(f"Chi phí: ${cost:.6f}")
```

DEMO 1 · BƯỚC 4

Không sợ provider sập

router.py

Cân bằng tải: 2 deployment cùng tên "chat" chia nhau gánh request

Fallback: "chat" lỗi thì tự chuyển sang "backup"

num_retries=2: thử lại trước khi fallback

Azure tự đọc AZURE_API_KEY, AZURE_API_BASE, AZURE_API_VERSION từ biến môi trường.

```
from litellm import Router
router = Router(
    model_list=[
        {"model_name": "chat",
         "litellm_params": {"model": "azure/gpt-4o"}},
        {"model_name": "chat",
         "litellm_params": {"model": "openai/gpt-4o"}},
        {"model_name": "backup",
         "litellm_params": {"model": "anthropic/claude-sonnet-5-5"}},
    ],
    fallbacks=[{"chat": ["backup"]}],
    num_retries=2,
)
r = router.completion(model="chat", messages=msgs)
```

Demo 2

AI Gateway cho doanh nghiệp

config.yaml · chạy Gateway · virtual key · dashboard

DEMO 2 · BƯỚC 1

Khai báo model trong config.yaml

config.yaml

model_list: đặt tên "thân thiện" cho từng model

os.environ/...: key gốc nằm trên server, app không bao giờ thấy

database_url: cần Postgres để lưu virtual key và chi phí

```yaml
model_list:
  - model_name: gpt-4o
    litellm_params:
      model: openai/gpt-4o
      api_key: os.environ/OPENAI_API_KEY
  - model_name: claude
    litellm_params:
      model: anthropic/claude-sonnet-5-5
      api_key: os.environ/ANTHROPIC_API_KEY

litellm_settings:
  fallbacks: [{"gpt-4o": ["claude"]}]

general_settings:
  master_key: sk-1234      # khóa admin
  database_url: os.environ/DATABASE_URL
```

DEMO 2 · BƯỚC 2

Postgres: bộ nhớ của Gateway

POSTGRES LƯU GÌ?

- **Virtual key, team, user**
- **Chi phí từng request**
- **Log để audit**
- **Tài khoản Admin UI**

Không có DB? Gateway vẫn chạy nhưng chỉ có master key: không tạo key, không ngân sách, không giao diện web.

Bảng dữ liệu do LiteLLM tự tạo khi khởi động.

```yaml
# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: litellm
      POSTGRES_PASSWORD: pw
      POSTGRES_DB: litellm
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]  # tắt máy không mất
volumes:
  pgdata:
```

```
docker compose up -d
export DATABASE_URL="postgresql://litellm:pw@localhost:5432/litellm"
```

DEMO 2 · BƯỚC 3

Chạy Gateway, app cũ giữ nguyên

`terminal + app.py`

App đang dùng thư viện OpenAI?
Chỉ đổi base_url và api_key.

model= tên trong config.yaml,
không cần biết hãng nào đứng sau

```
# DATABASE_URL đã đặt ở bước 2
pip install 'litellm[extra_proxy]'

prisma generate --schema="$(python -c "import litellm, os; print(os.path.join(os.path

litellm --config config.yaml
# → Gateway chạy tại http://localhost:4000
```

```
from openai import OpenAI      # vẫn là thư viện OpenAI!

client = OpenAI(
    api_key="sk-team-...",             # virtual key của team
    base_url="http://localhost:4000",  # trỏ về Gateway
)
r = client.chat.completions.create(
    model="claude",                    # tên trong config.yaml
    messages=[{"role": "user", "content": "Xin chào!"}],
)
```

DEMO 2 · BƯỚC 4

Virtual key và ngân sách theo team

Admin API

Mỗi team một key: giới hạn model, ngân sách, thời hạn

Hết $50: key tự bị chặn, không lo "cháy" hóa đơn

Xem trực quan: dashboard tại localhost:4000/ui

```
# Admin tạo key cho team Marketing: chỉ dùng gpt-4o, tối đa $50
curl -X POST http://localhost:4000/key/generate \
  -H "Authorization: Bearer sk-1234" \
  -H "Content-Type: application/json" \
  -d '{"models": ["gpt-4o"],
       "max_budget": 50,
       "duration": "30d",
       "metadata": {"team": "marketing"}}'
# → {"key": "sk-abc...", "expires": "...", ...}

# Xem team đó đã tiêu bao nhiêu
curl "http://localhost:4000/key/info?key=sk-abc..." \
  -H "Authorization: Bearer sk-1234"
```

TÓM TẮT CHO DOANH NGHIỆP

Mỗi nỗi đau, một tính năng

Nỗi đau	LiteLLM giải quyết bằng
Lộ API key gốc cho từng team	Virtual key: cấp, thu hồi, giới hạn theo team và dự án
Không biết ai tiêu bao nhiêu	Theo dõi chi phí theo key, user, team
Một hãng sập là app sập	Retry, fallback, cân bằng tải giữa các deployment
Cần kiểm soát nội dung vào/ra	Guardrails cấu hình theo dự án
Cần log để audit, đánh giá	Callback sang Langfuse, MLflow, Lunary...

KHÔNG CHỈ CHATBOT

Gateway cho cả hệ thống agent

MCP Gateway

Kết nối MCP server với bất kỳ LLM nào, quản lý tập trung.

Agent A2A

Gọi agent từ LangGraph, Vertex AI Agent Engine, Bedrock AgentCore, Pydantic AI...

Agent harness

Chạy Claude Code, Codex, OpenCode, Deep Agents trên bất kỳ model nào.

LLM Gateway đang thành "trạm thu phí" cho mọi lời gọi AI trong tổ chức.

SO SÁNH NHANH

LiteLLM hay 9Router?

Tiêu chí	LiteLLM	9Router
Dành cho	Tổ chức nhiều team, nhiều app	Dev dùng Claude Code, Cursor, Codex...
Cài đặt	pip hoặc Docker, tự host server	npm install -g 9router, chạy local
Nhà cung cấp	100+, gọi qua API key	60+, cả gói subscription và free tier
Fallback	Retry, fallback, cân bằng tải	3 tầng: subscription → giá rẻ → miễn phí
Quản trị	Virtual key, ngân sách, guardrails	Theo dõi quota, gộp nhiều tài khoản
Điểm riêng	MCP/A2A Gateway, SDK Python	Nén token, MITM bridge cho IDE

Tóm lại: 9Router giúp cá nhân tận dụng gói có sẵn và tiết kiệm tiền; LiteLLM là hạ tầng có quản trị cho doanh nghiệp.

Lưu ý: chuyển lưu lượng của gói subscription sang công cụ khác có thể trái điều khoản nhà cung cấp; chính 9Router cũng khuyên kiểm tra chính sách từng công cụ.

CÓ NÊN DÙNG?

Khi nào cần, khi nào chưa

Chưa cần vội

- Dự án nhỏ, chỉ dùng 1 model
- Prototype vài ngày rồi bỏ
- Chưa có ai lo vận hành thêm 1 dịch vụ

Nên dùng ngay

- Nhiều team, nhiều model
- Cần quản trị chi phí và quyền truy cập
- Cần audit, tuân thủ (ngân hàng, tài chính...)
- Muốn thử model mới mà không sửa app

MÌ AI · HỌC AI THEO CÁCH MÌ ĂN LIỀN

Thank you!

Mã nguồn để dưới phần mô tả của video.

github.com/BerriAI/litellm

miai.vn